

```

"""
real_time_detection.py
-----
Runs real-time facial expression detection using a
webcam feed.

Pipeline:
    1. Capture video frames with OpenCV.
    2. Detect faces in each frame with a Haar Cascade
       classifier.
    3. Crop, resize (48x48) and normalize each face.
    4. Feed the face into the trained CNN
       (model/emotion_model.h5).
    5. Overlay the predicted emotion label and
       confidence on the frame.

Controls:
    q -> quit

Usage:
    python real_time_detection.py
"""

import os
import cv2
import numpy as np
from tensorflow.keras.models import load_model

MODEL_PATH = os.path.join("model", "emotion_model.h5")
IMG_SIZE = 48

EMOTIONS = ["Angry", "Disgust", "Fear", "Happy",
            "Sad", "Surprise", "Neutral"]

# BGR colors per emotion for on-screen overlay
COLORS = {
    "Angry": (0, 0, 255),
    "Disgust": (0, 128, 0),

```

```

    "Fear": (128, 0, 128),
    "Happy": (0, 255, 255),
    "Sad": (255, 0, 0),
    "Surprise": (0, 165, 255),
    "Neutral": (200, 200, 200),
}

def load_face_detector():
    cascade_path = cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
    detector = cv2.CascadeClassifier(cascade_path)
    if detector.empty():
        raise IOError("Failed to load Haar Cascade
face detector.")
    return detector

def load_emotion_model(path):
    if not os.path.exists(path):
        raise FileNotFoundError(
            f"'{path}' not found. Run train_model.py
first, or place a "
            f"pretrained emotion_model.h5 in the
'model/' folder."
        )
    return load_model(path)

def preprocess_face(gray_frame, x, y, w, h):
    face = gray_frame[y:y + h, x:x + w]
    face = cv2.resize(face, (IMG_SIZE, IMG_SIZE))
    face = face.astype("float32") / 255.0
    face = np.expand_dims(face, axis=(0, -1)) # shape
(1, 48, 48, 1)
    return face

def main():
    face_detector = load_face_detector()
    model = load_emotion_model(MODEL_PATH)

```

```

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise IOError("Cannot open webcam. Check that
it is connected and not in use.")

print("Starting real-time detection. Press 'q' to
quit.")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame from webcam.")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detector.detectMultiScale(
        gray, scaleFactor=1.1, minNeighbors=5,
minSize=(48, 48)
    )

    for (x, y, w, h) in faces:
        face_input = preprocess_face(gray, x, y,
w, h)
        predictions = model.predict(face_input,
verbose=0)[0]
        emotion_idx = int(np.argmax(predictions))
        emotion = EMOTIONS[emotion_idx]
        confidence =
float(predictions[emotion_idx]) * 100
        color = COLORS.get(emotion, (255, 255,
255))

        cv2.rectangle(frame, (x, y), (x + w, y +
h), color, 2)
        label = f"{emotion} ({confidence:.1f}%)"
        cv2.putText(
            frame, label, (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, color,
2
        )

    cv2.imshow("Real-Time Facial Expression

```

```
Detection", frame)

        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```